

Tic-Tac-Toe Using Minimax Algorithm

1. Aim

To implement a **Tic-Tac-Toe game using the Minimax algorithm** so that an intelligent computer player can select the best possible move based on the current game state.

2. Problem Statement

Tic-Tac-Toe is a two-player game played on a **3×3 grid**, where one player uses X and the other uses O. The objective is to place three identical symbols in a row, column, or diagonal. The problem is to develop an intelligent agent that can analyze the possible game states and select the optimal move against the opponent.

3. Solution

The problem is solved using the **Minimax algorithm**, which explores possible future moves and evaluates the resulting game states. The computer acts as the maximizing player and tries to achieve the highest score, while the opponent is considered the minimizing player. By evaluating all possible moves, the algorithm selects the best available move for the computer.

4. Algorithm

1. Initialize an empty 3×3 Tic-Tac-Toe board.
2. Allow the human player and computer to make alternate moves.

3. Check whether the current state is a winning, losing, or draw state.
4. Generate all possible moves for the computer.
5. Apply each possible move temporarily.
6. Use the Minimax algorithm to evaluate the resulting game state.
7. Assign higher scores to winning states and lower scores to losing states.
8. Select the move with the highest score for the computer.
9. Continue the game until a player wins or the board becomes full.
10. Display the final game result.

5. Program

```
import math
```

```
board = [" " for _ in range(9)]
```

```
def print_board():
```

```
    print()
```

```
    for i in range(0, 9, 3):
```

```
    print(board[i], "|", board[i+1], "|", board[i+2])
    if i < 6:
        print("--+---+--")
print()
```

```
def check_winner():
    winning_positions = [
        (0, 1, 2),
        (3, 4, 5),
        (6, 7, 8),
        (0, 3, 6),
        (1, 4, 7),
        (2, 5, 8),
        (0, 4, 8),
        (2, 4, 6)
    ]
```

```
    for a, b, c in winning_positions:
        if board[a] == board[b] == board[c] != " ":
            return board[a]
```

```
if " " not in board:
```

```
    return "Draw"
```

```
return None
```

```
def minimax(is_maximizing):
```

```
    result = check_winner()
```

```
    if result == "O":
```

```
        return 1
```

```
    if result == "X":
```

```
        return -1
```

```
    if result == "Draw":
```

```
        return 0
```

```
    if is_maximizing:
```

```
        best_score = -math.inf
```

```
        for i in range(9):
```

```
        if board[i] == " ":
            board[i] = "O"
            score = minimax(False)
            board[i] = " "
            best_score = max(best_score, score)

    return best_score

else:
    best_score = math.inf

    for i in range(9):
        if board[i] == " ":
            board[i] = "X"
            score = minimax(True)
            board[i] = " "
            best_score = min(best_score, score)

    return best_score
```

```
def best_move():
    best_score = -math.inf
    move = None

    for i in range(9):
        if board[i] == " ":
            board[i] = "O"
            score = minimax(False)
            board[i] = " "

            if score > best_score:
                best_score = score
                move = i

    return move
```

```
print("Tic-Tac-Toe")
print("You are X, Computer is O")
```

```
while True:
```

```
print_board()

# Human move
position = int(input("Enter position (1-9): ")) - 1

if position < 0 or position > 8 or board[position] != " ":
    print("Invalid move. Try again.")
    continue

board[position] = "X"

result = check_winner()

if result:
    print_board()
    print("Result:", result)
    break

# Computer move
computer_move = best_move()
```

```
board[computer_move] = "O"
```

```
result = check_winner()
```

```
if result:
```

```
    print_board()
```

```
    print("Result:", result)
```

```
    break
```

6. Result

The Tic-Tac-Toe game was successfully implemented using the **Minimax algorithm**. The computer player evaluates the possible moves and selects the optimal move based on the current game state. The program successfully allows a human player to compete against an intelligent computer player.

Sample Output:

Tic-Tac-Toe

You are X, Computer is O

```
X | |
```

```
--+---+--
```

```
| O |
```



```

--+---+--
|  |X

```

Enter position (1-9): 2

```

X|X|O
--+---+--
|O|
--+---+--
|  |X

```

Thus, the **Tic-Tac-Toe** game was successfully implemented using the Minimax algorithm, enabling the computer to make intelligent and optimal decisions during gameplay.

```

PythonProject10  Version Control  Current File  Run  TIC TAC TOE
TIC TAC TOE.py  H4 BETA DBL LINDO.py  DECISION TREE.py  FEED FORWARD NEURAL NETWORK.py  MAP CNN (LINDO).py  TIC TAC TOE.py  MINIMAX.py

Enter position (1-9): 3
O |  | X
--+---+--
| X |
--+---+--
|  |

Enter position (1-9): 7
O |  | X
--+---+--
| X |
--+---+--
O |  |

Enter position (1-9): 2
O | X | X
--+---+--
| X |
--+---+--
O |  |

Enter position (1-9): 4
O | X | X
--+---+--
O | X |
--+---+--
O |  |

O wins!
PythonProject10  TIC TAC TOE.py  30/7 (32 chars, 1 line break)  CRLF  UTF-8  4 spaces  Python 3.11 (PythonProject10)  20:55 06-09-2024

```